Traffic Flow Optimization System Using Graph Theory

Difficulty Level	Full
Suggested Language	C++
Maximum Team Size	3
Skill Set	C++, Data Structures, Algorithms

1. Introduction

Modern urban areas face severe traffic congestion due to increasing vehicle density and inefficient traffic control systems. Traditional traffic management relies on static signal timings and fixed routing, which fail to adapt to real-time conditions.

This project focuses on designing a **Traffic Flow Optimization System** that models a city's road network and dynamically optimizes vehicle movement using mathematical modelling and efficient data structures. The system integrates vehicle-level simulation, Congestion-aware routing, queue-based intersection modeling and adaptive traffic signal control

2. Problem Description

The objective is to design and implement a simulation-based system that:

- Models an urban road network as a graph
- Simulates vehicle movement and traffic flow
- Models congestion and queue formation at intersections
- Dynamically adjusts traffic signal timings
- Computes optimal routes for vehicles using current traffic states
- Minimizes congestion and travel time

The system should support multi-source traffic, real-time updates, congestion modelling, and adaptive optimization, while maintaining computational efficiency.

3. Solution Design (Tentative):

The system is designed as a **modular architecture**, where each class represents a specific physical or computational component derived from the mathematical model. The system operates in **discrete time steps (simulation cycles)**. At each simulation step:

1. Traffic flow is updated based on vehicle arrivals
2. Edge congestion and travel times are recalculated
3. Optimal routes are computed using shortest path algorithms
4. Traffic signals are adjusted dynamically
5. Performance metrics are recorded

This loop continues to simulate real-world traffic behavior.

4. Core System Components

4.1 Road Network Model

The road network is represented as a directed graph:

$$G = (V, E)$$

Where:

- V: set of intersections (nodes)
- E: set of roads (edges)

Each edge e_{ij} represents a road from node i to node j. Every road segment is associated with the following parameters:

- l_{ij}: Length of the road segment, representing the physical distance between intersections

- v_{ij}^{max}: Maximum allowed speed on the road, determining how fast vehicles can travel under ideal conditions
- c_{ij}: Capacity of the road, defined as the maximum number of vehicles that can occupy the road at any given time
- $f_{ij}(t)$: Number of vehicles currently traveling on the road at time t
- $Q_{ij}(t)$: Number of vehicles waiting at the downstream intersection $j$after completing the road
- $w_{ij}(t)$: Travel time required to traverse the road at time t, which depends on congestion

4.2 Traffic Flow Model Link Volume

Traffic on each road changes over time as:

$$f_{ij}(t+1) = f_{ij}(t) + a_{ij}(t) - x_{ij}(t)$$

Where:

- $a_{ij}(t)$: Vehicles entering the edge e_{ij}
- $x_{ij}(t)$: Vehicles leaving the edge e_{ij} and entering into queue Q_{ij}

When vehicles reach an intersection but cannot proceed immediately due to red signal or congestion at incoming edge, they form queues. The queue evolves as:

$$Q_{ij}(t+1) = Q_{ij}(t) + x_{ij}(t) - d_{ij}(t)$$

Where:

- $d_{ij}(t)$: Vehicles leaving the queue at edge e_{ij}

Movement from one road to another is not unrestricted. It is governed by physical and operational constraints that reflect real traffic behavior.

$$d_{ij}(t) = g_{ij}(t) \cdot \min\left(Q_{ij}(t), \mu_{ij}, c_{jk} - f_{jk}(t)\right)$$

Where:

- $g_{ij}(t)$: signal state (1 = green, 0 = red)
- μ_{ij}: maximum discharge rate
- $c_{jk} - f_{jk}(t)$: available capacity in next edge

4.3 Congestion Model

Congestion level is defined as:

$$\rho_{ij} = \frac{f_{ij}}{c_{ij}}$$

- $\rho \approx 0$: free road
- $\rho \approx 1$: congested road

4.4 Travel Time Model

Travel time of roads increases with congestion using a nonlinear function: a = 0.15, b = 4

$$w_{ij}(t) = w_{ij}^{free}\left(1+\alpha\left(\frac{f_{ij}(t)}{c_{ij}}\right)^{\beta}\right)$$

Where:

- α: congestion sensitivity
- β: nonlinearity factor
- w_{ij}^{free}: ideal travel time with no congestion.

$$w_{ij}^{free} = \frac{l_{ij}}{v_{ij}^{max}}$$

4.5 Multi-Source Multi-Destination Routing

- Vehicles are generated from multiple sources simultaneously
- Each has its own destination

$$S = \{s_1, s_2, \ldots, s_k\}$$
$$D = \{d_1, d_2, \ldots, d_m\}$$

Objective:

$$\textbf{min}\ (\sum \boldsymbol{T_{sd}})$$

Where:

- T_{sd}: travel time for a vehicle going from source s to destination d.

The system handles simultaneous routing and flow for all vehicles.

4.6 Vehicle Model

Each vehicle is treated as an independent entity with:

- Source s_v
- Destination d_v
- Current edge
- Remaining travel time $r_v(t)$
- Overall status (moving, waiting, or arrived).

When a vehicle enters a road, its remaining travel time $r_v(t)$ is initialized to the road's current travel time $w_{ij}(t)$, which accounts for congestion. At each simulation time step, remaining travel time is updated as:

$$r_v(t+1) = r_v(t) - 1$$

When:

$$r_v(t) = 0$$

the vehicle reaches the intersection and contributes to $x_{ij}(t)$.

4.7 Routing Model (Shortest Path)

Vehicles select routes dynamically based on current traffic conditions.

Edge Cost

$$cost(e_{ij}) = w_{ij}(t)$$

Shortest Path

$$P_v = \arg\ \min\ \sum w_{ij}(t)$$

Where:

- P_v: The path taken by a vehicle from source to destination

This is computed using **Dijkstra's Algorithm**.

4.8 Traffic Signal Model

Traffic signals regulate which roads are allowed to release vehicles at each time step. Each intersection controls multiple incoming roads by giving the green signal to the road with the longest queue.

$$g_{ij}(t) \in \{0,1\}$$

Where:

- 1 = green signal
- 0 = red signal

$g_{ij}(t) = 1$ for edge with maximum $Q_{ij}(t)$

Signal Constraint:

$$\sum_{(i,j)\in In(j)} g_{ij}\,(t) = 1$$

Only one incoming road at an intersection can move at a time.

At each simulation step, vehicles on the green road move to the next segment, while vehicles on red roads wait in queue , increasing congestion. Traffic signals operate in discrete time steps synchronized with the vehicle simulation. A signal remains green for a fixed duration (Tg steps) or until the queue reduces below a threshold, then the next road with the longest queue is activated.

4.9 Objective Function (Goal of the System)

The system seeks to minimize congestion and delay across the entire network:

$$\min \sum_{t} \sum_{(i,j)} \left[\alpha Q_{ij}(t) + \beta \left(\frac{f_{ij}(t)}{c_{ij}} \right)^2 \right]$$

This balances:

- Queue lengths (waiting time)
- Congestion levels (road utilization)

4.10 Performance Metrics

The effectiveness of the system is evaluated using:

- **Average Travel Time**

$$\frac{1}{N} \sum T_{sd}$$

- **Total Delay**

$$\sum (T_{sd} - T_{sd}{}^{free})$$

- **Throughput**
  Number of vehicles reaching destination per unit time
- **Average Congestion Level**

$$\frac{1}{|E|} \sum \frac{f_{ij}(t)}{c_{ij}}$$

COMPLEX ENGINEERING PROBLEM ASSESSMENT RUBRICS

Trait	Exceptional [10-8]	Acceptable [7-6]	Amateur [5-3]	Unsatisfactory [2-0]
Problem Understanding 15%	The depth of the problem is exceptionally understood.	The depth of problem is understood to a sufficient level.	The depth of the problem is very weak.	There is no knowledge of the problem depth.
Application Functionality 20%	Application compiles with no warnings. Robust operation of the application, with good recovery.	Application compiles with few or no warnings. Consideration given to unusual conditions with reasonable recovery	Application compiles and runs without crashing. Some attempt at detecting and correcting errors.	Application does not compile or compiles but crashes. Confusing. Little or no error detection or correction.
Specification & Data structure implementatio n 30%	The program works and meets all of the specifications. Welldesigned and Excellent implementation of data structure. Providing all the required functions that might be useful and aids in performing the operations on data structure.	The program works and produces the correct results and displays them correctly. It also meets most of the other specifications. Good implementation of data structure.	The program produces correct results but does not display them correctly. Fair implementation of data structure.	The program is producing incorrect results. Worst implementation of data structure.
Reusability 10%	The code could be reused as a whole or each class could be reused.	Most of the code could be reused in other programs.	Some parts of the code require change before they could be reused in other programs.	The code is not organized for reusability.
Efficiency 10%	The code is extremely efficient without sacrificing readability and understanding. No memory Leakage, Dangling pointers and bad pointers	The code is fairly efficient without sacrificing readability and understanding. Small memory Leakage, dangling pointers or bad pointers	The code is brute force and unnecessarily long. Inefficiently using pointers & dynamic memory.	The code is huge and appears to be patched together and hasn’t managed any memory leaks, dangling pointers and bad pointers.

Input Validation 5%	All the inputs entered by user are properly validated, a message displayed to user in case of wrong input entered by user and prompt till user enters the correct input.	Error Message displayed to user in case of in correct input entered by user. The program continues execution without prompting again till the correct input is entered	Some of the inputs are validated leaving the others.	No input validation.
Report (Documentation) 10%	The documentation is well written and clearly explains what the code is accomplishing and how.	The documentation consists of embedded comment and some simple header documentation that is somewhat useful in understanding the code.	The documentation is simply comments embedded in the code with some simple header comments separating routines.	The documentation is simply comments embedded in the code and does not help the reader understand the code.
Plagiarism	No Plagiarism			Plagiarized Code

Total Marks = O$\boldsymbol{btained\ Marks}$ $(\sum_{1}^{7} \boldsymbol{R_i}) * \boldsymbol{R_8}$